Italy Municipality Heat Map

Complete Implementation Package

Forecasting property appreciation, rental ROI, and euro/sq. m. valuations by Italian municipality

Prepared for	Pete - Italian property analytics app planning
Design basis	Reuses the attached Italian Properties architecture where it fits: Next.js App Router, TypeScript, Supabase PostgreSQL, Leaflet, Vercel, Vercel Blob, and TypeScript-based ingestion patterns.

This document serves as the consolidated implementation package for the MVP. It combines the product definition, functional requirements, technical architecture, data-source plan, schema design, API plan, front-end structure, model and back-testing framework, delivery roadmap, and implementation handoff materials needed for a developer to begin work.

1. Executive Summary

This application should be built as an analytics-first map product rather than a listing portal. The core outputs are: current residential valuation in euro per square meter, 12-month appreciation forecast, 12-month rental ROI forecast, and a confidence or data-quality score for each municipality.

The best implementation path is to reuse the current Italian Properties app platform where possible and extend it for time-series, spatial analytics, and forecasting. That means keeping Next.js, TypeScript, Supabase, Leaflet, Vercel, and the existing ingestion style, while adding PostGIS, a structured feature store, and Python batch jobs for model training and scoring.

For the MVP, use municipality as the public map unit and keep the schema ready for OMI-zone drilldown later.

2. Recommended Technology Stack

Layer	Recommended	Reuse	Notes
Front end	Next.js App Router + React + TypeScript	Yes	Strong fit for analytics UI, filters, map pages, rankings, and detail views.
Map	Leaflet	Yes	Good for MVP choropleths and municipality drilldowns.
Database	Supabase PostgreSQL + PostGIS	Extend	Keep Supabase; add spatial support and analytics schemas.
API	Next.js Route Handlers	Yes	Matches current application pattern.
Object storage	Vercel Blob	Yes	Use for exports, cached GeoJSON, and artifacts.
Ingestion	TypeScript ETL scripts	Extend	Keep current pattern for official datasets and controlled feeds.
Modeling	Python batch jobs	New	Best fit for feature engineering, forecasting, backtesting, and intervals.
Deployment	Vercel + Supabase + worker service	Extend	Add scheduled workers for ingestion and model runs.

3. Reuse vs. New Build Decisions

Reuse directly

- Next.js app framework and routing patterns.
- TypeScript codebase conventions and existing data access style.
- Supabase for the primary application database.
- Leaflet for municipality choropleths in the MVP.
- Vercel deployment and Vercel Blob for exported artifacts.

Extend materially

- Upgrade the current database design from listing-centric records to municipality and time-series analytics tables.
- Add PostGIS for boundary geometries, neighbor logic, and spatial joins.
- Broaden ingestion from scraper-only patterns to multi-source ETL for OMI, ISTAT, rental feeds, and regulation datasets.
- Add feature-store and forecast tables so the application serves scored municipalities rather than raw listings.

Add new capability

- Python feature engineering and forecasting jobs.
- Model run audit tables and publishable forecast snapshots.
- Confidence scoring, risk flags, and quality controls.

4. High-Level System Architecture

- Front end: Next.js pages for map, rankings, municipality detail, compare, methodology, and admin.
- Server layer: route handlers that query latest published forecast views and return map-friendly payloads.
- Database: Supabase PostgreSQL with PostGIS, organized into core, raw, mart, model, and admin schemas.
- Data and ML layer: batch ingestion, feature generation, model training, forecast scoring, and publish step.
- Storage: Vercel Blob for exports, cached GeoJSON snapshots, and downloadable methodology files.

5. Functional Modules

Module	Role	Key responsibilities
Geography & map service	Spatial serving	Municipality geometries, choropleth values, boundary simplification, neighbor relationships, and map-ready GeoJSON.
Data ingestion	Source loading	OMI values, OMI transactions, ISTAT population, ISTAT tourism, rental sources, and manual regulation tables.
Feature store	Derived analytics	Time-series trends, peer-relative metrics, spatial lags, liquidity indicators, and confidence inputs.
Forecast engine	Predictive scoring	Appreciation model, rental yield model, confidence intervals, versioning, and writeback.
Ranking engine	Decision support	Opportunity score, investor strategy profiles, and risk-adjusted ranks.
Admin console	Operations	Ingestion runs, failed rows, source audits, manual overrides, and model-run history.

6. Database and Schema Design

Canonical grain

- Primary public grain: municipality (comune) using the ISTAT municipality code.
- Secondary drilldown grain: OMI zone, stored now so the map can support future zone-level analysis.
- Time support: month, year, and semester periods, because values and transactions are semestral, tourism is monthly, and demographics are annual.

Core schemas

Schema	Purpose	Representative tables
core	Reference dimensions	regions, provinces, municipalities, municipality_neighbors, omi_zones, time_periods
raw	Source fidelity	omi_property_values, omi_transactions, istat_population, istat_tourism, rental_long_term, rental_short_term, regulatory_notes
mart	Curated analytics marts	municipality_values_semester, municipality_transactions_semester, municipality_demographics_year, municipality_tourism_month, municipality_rents_month
model	Feature store and forecasts	features_municipality_semester, forecasts_municipality
admin	Operational audit	ingestion_runs, model_runs

Key table catalog

- core.municipalities: One row per municipality; stores ISTAT code, province, region, flags, and boundary geometries.
- core.municipality_neighbors: Stores contiguity and distance metrics for spatial features.
- core.omi_zones: Stores OMI zone identifiers and geometries for future drilldown.
- mart.municipality_values_semester: Curated semestral property values by municipality and property segment.
- mart.municipality_transactions_semester: Curated semestral transaction counts and turnover indicators.
- mart.municipality_demographics_year: Annual population and household indicators.
- mart.municipality_tourism_month: Monthly arrivals, presences, foreign share, and tourism intensity.
- mart.municipality_rents_month: Monthly long-term and optional short-term rental indicators.
- model.features_municipality_semester: Feature store used for training and scoring.
- model.forecasts_municipality: Published forecast outputs, scores, drivers, and risk flags.

Representative schema fields

Table	Primary key	Important fields
core.municipalities	municipality_id	municipality_name, province_code, region_code, coastal_flag, mountain_flag, geom, geom_simplified
mart.municipality_values_semester	(municipality_id, semester_id, property_segment)	value_min_eur_sqm, value_max_eur_sqm, value_mid_eur_sqm, quality_grade
mart.municipality_transact	(municipality_id, semester_id,	transaction_count, turnover_rate, quality_grade

ions_semester	market_sector)	
mart.municipality_tourism_month	(municipality_id, month_id)	arrivals_total, presences_total, tourism_intensity
model.features_municipality_semester	(municipality_id, semester_id, property_segment, feature_version)	trend metrics, liquidity metrics, demographics, tourism, yields, spatial lags, risk scores, targets
model.forecasts_municipality	(municipality_id, forecast_date, horizon_months, property_segment, model_version)	forecast_appreciation_pct, forecast_gross_yield_pct, confidence bounds, opportunity_score, drivers, risks

7. Feature Store Design

Feature family	Representative features
Valuation	current_value_mid_eur_sqm, value_yoy_pct, value_2y_cagr, value_5y_cagr, value_volatility_8s, value_vs_peers
Liquidity and transactions	transactions_yoy_pct, transactions_per_1000_residents, turnover_rate, liquidity_score, thin_market_flag
Demographics	population_yoy_pct, population_5y_cagr, households_yoy_pct, working_age_share, old_age_index, foreign_resident_share
Tourism	tourism_intensity, tourism_yoy_pct, foreign_presences_share, seasonality_ratio, shoulder-season strength
Rental	long_term_rent_eur_sqm_month, rent_yoy_pct, gross_yield_long_term, short_term_adr_eur, occupancy_rate, gross_yield_short_term
Spatial	neighbor_mean_value, neighbor_mean_growth, travel-time/accessibility measures, coastal and mountain flags
Risk and confidence	regulatory_risk_score, seasonality_risk_score, demographic_risk_score, data_sparsity_score, confidence_input_score

Targets

- Appreciation target: 12-month percentage change in municipality residential euro per square meter.
- Rental target: forecast gross long-term rental yield over the next 12 months, with optional net yield after cost assumptions.
- Optional later target: short-term-rental yield where destination data is sufficiently strong.

8. API Specification

Endpoint	Method	Purpose
/api/map/layer	GET	Returns choropleth values by selected metric, horizon, segment, and filters.
/api/municipality/[istatCode]	GET	Returns municipality detail with current values, forecasts, trends, scores, drivers, and risk flags.
/api/rankings	GET	Returns filtered ranking tables by strategy, region, confidence, risk, and liquidity thresholds.
/api/compare	GET	Returns side-by-side comparisons for selected municipalities.

/api/filters	GET	Returns valid regions, provinces, strategies, and property segments for UI filters.
/api/admin/ingest/*	POST	Protected endpoints to trigger ingestion jobs.
/api/admin/model/*	POST	Protected endpoints to build features, train models, and publish scoring runs.

9. Front-End Application Structure

- Homepage: map preview, top municipalities, and methodology teaser.
- Map page: full-screen choropleth with filters, metric toggle, and municipality drawer.
- Municipality detail page: trend charts for values, transactions, tourism, demographics, and rental signals.
- Rankings page: sortable, filterable ranking table.
- Compare page: side-by-side municipality comparisons.
- Methodology page: plain-language explanation of sources, features, model logic, and limitations.
- Admin page: ingestion status, failed rows, model runs, and manual overrides.

Component groups

- Map components: ChoroplethMap, MapLegend, MetricToggle, FilterSidebar, MunicipalityDrawer.
- Municipality components: MunicipalityHeader, ScoreCards, DriversAndRisks, TrendChart, PeerComparison.
- Shared UI: Badge, StatCard, SegmentedControl, and export controls.

10. Ingestion and Refresh Architecture

- TypeScript ingestion package with common utilities for logging, CSV parsing, Excel parsing, geospatial normalization, and upsert behavior.
- Source loaders grouped by domain: omi, istat, rents, regulations, and scheduled jobs.
- All runs written to admin.ingestion_runs with source version, rows loaded, rows rejected, and error notes.

Refresh cadence

Cadence	Sources	Typical tasks
Daily	Rental feeds, regulation notes	Load new rental observations, refresh regulation snapshots, and validate changes.
Monthly	Tourism, rents	Rebuild tourism and rental marts, refresh rankings, and republish cached map layers.
Semester	OMI values and transactions	Load new semestral value and transaction data, rebuild features, retrain models, and score all municipalities.

11. Modeling and Forecasting Architecture

- Use Python batch jobs for feature engineering, model training, backtesting, diagnostics, and scoring.
- Recommended MVP models: LightGBM or CatBoost regressors for appreciation and rental yield.
- Store model metadata and metrics in admin.model_runs.
- Write scored outputs into model.forecasts_municipality and publish the latest approved snapshot through a stable mart view.

Forecasting workflow

- Step 1: ingest raw source files into raw tables.
- Step 2: normalize and curate source data into mart tables.
- Step 3: build municipality-semester features in model.features_municipality_semester.
- Step 4: train or update model versions and record metrics.
- Step 5: score all municipalities for the target horizon.
- Step 6: publish the latest approved forecast snapshot for the application.

12. Security, Permissions, and Publication Controls

- Use Supabase Auth for internal users with roles such as admin, analyst, and editor.
- Keep raw, model, and admin tables private; expose only approved views or API endpoints.
- Protect ingestion and model-run actions behind internal admin routes.
- Use publication controls such as publishable_flag, quality_grade, and manual_review_flag before any forecast is surfaced publicly.

13. Map Rendering and Performance Strategy

- Keep Leaflet for the MVP and serve simplified municipality GeoJSON.
- Use region-level chunking or cached map layers so the browser does not load unnecessary full-detail geometry.
- Precompute simplified boundaries and map metrics to reduce server work per request.
- If the map later becomes too heavy, move to vector tiles; this is not required for the first release.

14. Quality Assurance and Data Governance

- Validate municipality identifiers against canonical ISTAT codes.
- Reject duplicate municipality-period-segment rows.
- Flag implausible jumps in euro per square meter or yield values.
- Suppress forecasts where data depth is below the minimum threshold.
- Widen confidence intervals for thin-liquidity municipalities.
- Maintain data quality fields such as quality_grade, data_sparsity_score, publishable_flag, and manual_review_flag.

15. Deployment Architecture

Layer	Service	Notes
Front end	Vercel	Hosts the Next.js application and public pages.
Database	Supabase Postgres + PostGIS	Stores reference, mart, feature, and forecast data.
Workers	Railway, Render, or GitHub Actions	Runs scheduled ingestion and Python model jobs.
Blob storage	Vercel Blob	Stores map cache files, exports, and print artifacts.

16. Delivery Roadmap

Phase	Objective	Main deliverables
1	Foundation	Enable PostGIS, load municipal boundaries, OMI values, OMI transactions, and core ISTAT datasets.
2	Analytics MVP	Build feature store, appreciation model, current valuation map, rankings, and municipality detail pages.
3	ROI MVP	Add long-term rental layer, rental yield forecast, compare page, and analyst tools.
4	Advanced expansion	Add short-term rental modeling, regulation scoring, and OMI-zone drilldown.

17. Final Recommendation

Build this as a new analytics product on top of the existing Italian Properties platform patterns, rather than as a separate greenfield stack. Reusing Next.js, TypeScript, Supabase, Leaflet, and Vercel keeps delivery fast and coherent, while adding PostGIS and Python model jobs gives the project the analytical depth it needs. The MVP should launch municipality-level current valuations, appreciation forecasts, and long-term rental ROI, with OMI-zone drilldown and short-term-rental intelligence reserved for later phases.

Appendix: Suggested project structure

- src/app, src/components, src/lib, src/types
- database/migrations, database/seeds, database/views
- ingestion/common, ingestion/omi, ingestion/istat, ingestion/rents, ingestion/regulations, ingestion/jobs
- ml/config, ml/features, ml/train, ml/score, ml/eval
- scripts/backfill, publish-map-cache, rebuild-rankings

Appendix B: Data Sources to Use for This Project

This appendix adds a practical source catalog for the municipality heat-map project. Use official Italian public data as the backbone of the MVP, then layer in portal or commercial rental datasets where public coverage is weak. For every source, store the source name, refresh cadence, access notes, source URL, and exact geographic grain used in the pipeline.

Core public and essential feeds

Source	Key data	Grain	Cadence	Access	Primary use	Priority
OMI (Osservatorio Mercato Immobiliare)	Official min/max EUR/sqm ranges by property type, OMI micro-zones, and long historical series.	Zona OMI, municipality	Semi-annual	Free public downloads	Baseline valuations, appreciation history, peer comparisons, and future OMI-zone drilldown.	Critical
Agenzia delle Entrate - OMI transaction volumes	Residential and market-sector transaction counts / activity by municipality and period.	Municipality	Semi-annual	Free public downloads	Liquidity, turnover, market depth, and price-versus-volume momentum features.	Critical
ISTAT resident population and census datasets	Population, households, age structure, density, foreign share, and demographic structure.	Municipality, year	Annual	Public datasets	Population growth, household formation, demographic stability, and affordability context.	Critical
ISTAT tourism survey	Monthly arrivals, presences, foreign share, and seasonality indicators.	Municipality, month	Monthly	Public datasets	Tourism intensity, seasonality, vacation-market demand, and STR demand signals.	High
ISTAT territorial bases and commuting matrices	Boundaries, contiguity, commuting flows, travel-time, and spatial context inputs.	Municipality, geometry	Periodic / annual	Public datasets	Map geometries, neighbor features, commuter integration, and accessibility scores.	High

Secondary and optional feeds

Source	Key data	Grain	Cadence	Access	Primary use	Priority
Agenzia delle Entrate - locazioni / market summaries	Rental-market summaries, counts, surfaces, and canoni where	Varies by release	Periodic	Public datasets	Cross-check rental assumptions and enrich long-term rental	Medium

	available.				modeling where coverage exists.	
Long-term rental portal or licensed feed	Asking rents, listing counts, property-type rent levels, and optional days on market.	Municipality or geo-cluster, month	Monthly or weekly	Commercial / scraped / partner feed	Gross yield estimation, rent trends, and long-term rental ROI forecasts.	Critical
Short-term rental provider	ADR, occupancy, RevPAR, active listings, seasonality, and listing supply.	Municipality or market cluster, month	Monthly	Commercial feed	Vacation-rental ROI layer for tourism-heavy municipalities.	Phase 2
Manual / curated regulatory dataset	Municipal STR restrictions, heritage rules, historic-center constraints, and local policy changes.	Municipality, effective date	As needed	Manual internal curation	Risk flags, regulation score, and analyst due-diligence overlays.	High
Reserved-access workflows (SPID / Sister / AdE services)	Title, cadastral, mortgage, and property-specific verification workflows.	Property / subject specific	On demand	Reserved access	Analyst mode and deal-level due diligence, not broad municipality forecasting ETL.	Optional

Recommended ingestion order

• OMI values and OMI transaction volumes should be the first ingestion pipeline because they anchor both price levels and liquidity.

• ISTAT population, tourism, and geography datasets should be the second wave because they provide the strongest explanatory variables for municipal forecasting.

• Long-term rental data is the most important non-public enrichment because it unlocks gross-yield and rental-ROI forecasts.

• Short-term rental feeds and local regulation tracking should be added after the long-term rental layer is stable.

• SPID-enabled and reserved-access services should support analyst workflows, title checks, and property due diligence, not the base municipality heat map.

Suggested source links

• OMI / Agenzia delle Entrate public area: https://www.agenziaentrate.gov.it/portale/web/guest/schede/fabbricatiterreni/omi

• ISTAT datasets and territorial statistics: https://www.istat.it/

Appendix C: Model Back-Testing and Fine-Tuning Framework

This appendix adds the back-testing, model-validation, and model-promotion plan that should sit alongside the forecasting architecture. The goal is to prove that the models are useful on past unseen periods, quantify where they are reliable or weak, and provide a disciplined path for tuning and promoting new model versions.

Objectives of the back-testing program

- Demonstrate that the appreciation and rental-yield models beat simple baselines such as flat forecasts, momentum carry-forward, and province or region averages.
- Measure not only point-forecast accuracy, but also directional accuracy, ranking usefulness, and confidence-interval calibration.
- Identify where the models work best and worst, such as high-liquidity municipalities, tourism-heavy municipalities, small rural markets, coastal areas, and data-sparse comuni.
- Create an evidence-based tuning loop so that new features, new data sources, and new model families can be promoted only when they show better out-of-sample performance.

Validation design

- Use walk-forward or rolling-origin back-testing rather than random train/test splits. Each historical forecast must be trained only on data that would have been available at that forecast date.
- At each forecast origin, train the model on all eligible history up to that semester, score the next 12-month horizon, and compare the prediction with the realized outcome 12 months later.
- Repeat the process over many historical forecast origins so the team can evaluate stability across market regimes rather than relying on a single holdout period.
- Store every back-test prediction so the team can review municipality-level misses, inspect regime changes, and compute aggregate metrics over time.

Recommended forecast targets to back-test

- Appreciation target: forward 12-month percentage change in municipality residential euro per square meter.
- Rental target: forward 12-month gross long-term rental yield, with optional supporting sub-targets such as rent growth and net yield after cost assumptions.
- Ranking target: whether municipalities that the system ranked highly at each historical forecast origin later outperformed lower-ranked municipalities over the same horizon.

Baseline models the system must beat

- Flat baseline: assume the next 12 months produce no change.
- Momentum baseline: carry the previous 12-month change forward into the next 12 months.
- Province baseline: use the province average for the municipality forecast.
- Region baseline: use the region average for the municipality forecast.

- Yield persistence baseline: assume current gross yield or recent rent trend persists into the next period.

Evaluation metrics

- Numeric forecast accuracy: MAE, RMSE, median absolute error, sMAPE where appropriate, and bias.
- Directional usefulness: share of forecasts that correctly called the direction of appreciation or rent growth.
- Ranking value: Spearman rank correlation, top-quintile and top-decile realized performance, and spread between top-ranked and bottom-ranked groups.
- Calibration: interval coverage rate, interval width, and realized error by confidence bucket.

Back-test segmentation cuts

- Geography: region, province, north/center/south/islands, coastal versus inland, and mountain versus non-mountain municipalities.
- Market structure: high-liquidity versus thin-liquidity municipalities, low-price versus high-price municipalities, and tourism-heavy versus ordinary residential municipalities.
- Data quality: strong direct coverage versus inferred coverage, and municipalities grouped by quality grade or data sparsity score.
- Population and demand profile: growing municipalities, stable municipalities, and population-decline municipalities.

Fine-tuning and model improvement loop

- Run feature-family ablation tests so the team can measure the incremental value of tourism, demographics, spatial lag, supply, rental, and liquidity features.
- Compare model families, such as linear baselines, gradient boosting, spatially enriched models, hierarchical models, and ensembles.
- Tune separately by horizon where necessary. A 12-month appreciation model may differ from a 24-month model, and rental forecasting may need a different configuration than price forecasting.
- Consider segmented production models where justified, for example one configuration for major urban markets, one for tourism-heavy municipalities, and one for thin rural markets.
- Use historical residuals and back-test error patterns to calibrate confidence bands rather than assigning confidence scores subjectively.

Operational workflow to add to the build

- Create a repeatable back-test job that trains on a historical cutoff, scores the next horizon, and writes municipality-level predictions and realized outcomes to dedicated evaluation tables.
- Generate a summary report for every model version showing baseline comparisons, aggregate metrics, segment cuts, and ranking-bucket performance.
- Define model-promotion gates. A model version should only be promoted if it beats agreed baselines, does not materially worsen calibration, and behaves acceptably in weak-data segments.

- Run a full back-test whenever a new major model version is proposed, and rerun a lighter monitoring package when new semestral OMI data arrives.

Recommended database additions

- model.backtest_runs: one record per historical evaluation cycle, with forecast origin, target horizon, status, and model version.
- model.backtest_predictions: municipality-level predicted and realized values for each back-test run.
- model.backtest_summary: aggregate metrics by run, geography, market segment, and data-quality segment.
- model.model_comparisons: side-by-side metrics for baseline models and candidate production models, supporting promotion decisions.

Recommended promotion criteria

- Beat flat and province baselines on core numeric metrics for the main forecast targets.
- Show useful ranking separation, such as a clear realized-performance spread between the top and bottom ranked groups.
- Demonstrate that higher-confidence municipalities truly produce lower realized error than low-confidence municipalities.
- Avoid strong systematic bias in weak markets, such as chronic over-forecasting in thin-liquidity or highly seasonal municipalities.

Public methodology guidance

- Publish a concise methodology note stating that models are validated using walk-forward historical testing rather than random splits.
- Disclose a small set of understandable performance figures, such as back-tested MAE, direction accuracy, and top-quintile outperformance.
- Surface confidence scores, data-quality grades, and risk flags at the municipality level so users can see where the system is more or less reliable.

15. Implementation Package Deliverables

This section defines what should be treated as the formal implementation package for handoff to engineering. The goal is to remove ambiguity so a developer can start building the MVP without needing to redesign the product, data model, or validation approach.

The package should be divided into required, recommended, and optional assets. Required items are necessary to begin build work. Recommended items materially reduce delivery risk. Optional items accelerate implementation but are not blockers for the MVP.

Deliverable	Status	Purpose	Primary users
Product definition and MVP scope	Required	Defines what is being built, what is deferred, and how success is measured.	Founder, product, engineering
Functional requirements	Required	Specifies map behavior, filters, pages, admin actions, and outputs.	Product, engineering, QA
Technical architecture	Required	Defines stack, platform reuse, services, jobs, storage, auth, and deployment.	Engineering
Database schema and migrations	Required	Provides build-ready table definitions, relationships, indexes, and serving views.	Engineering, data
Data-source and ingestion plan	Required	Maps every source to refresh cadence, fields used, and target tables.	Engineering, data
API contract package	Required	Lets front-end and back-end teams build against fixed request and response shapes.	Engineering
Modeling and back-testing framework	Required	Defines targets, features, baselines, evaluation metrics, and promotion gates.	Data science, engineering
Front-end wireframes and component map	Recommended	Reduces UI ambiguity for the map, municipality page, rankings, compare, and methodology views.	Design, engineering
Environment, deployment, and CI/CD notes	Recommended	Speeds local setup, staging, production deployment, and	Engineering, DevOps

		scheduled jobs.	
Starter scaffolding repository	Optional	Includes route stubs, migration files, and job placeholders to accelerate build-out.	Engineering

Package file list

The implementation package should be delivered as a structured folder of printable documents plus machine-usable technical assets. Recommended top-level files include: 01-product-definition.pdf/docx, 02-functional-requirements.pdf/docx, 03-technical-architecture.pdf/docx, 04-data-sources-and-ingestion.pdf/docx, 05-database-schema.sql, 06-erd.pdf, 07-api-contracts.pdf or yaml, 08-modeling-and-backtesting.pdf/docx, 09-frontend-wireframes.pdf, 10-deployment-and-environment.pdf/docx, and 11-build-roadmap.pdf/docx.

MVP acceptance criteria

The MVP should be considered build-ready when the package clearly defines the municipality heat map, current euro/sq. m. valuation layer, 12-month appreciation forecast, long-term rental ROI forecast, municipality detail page, rankings page, compare page, methodology page, confidence scoring, and internal back-testing workflow.

Handoff checklist

Before development begins, the team should confirm that the source inventory is frozen for the first build, ISTAT code matching rules are documented, database migrations run cleanly in a fresh environment, endpoints are named and versioned, model targets are defined without ambiguity, and publish or suppress rules are stated for sparse municipalities.

Recommended next technical assets

After this package, the fastest path into execution is to generate the following implementation artifacts: production SQL migration files, Supabase PostGIS enablement steps, Next.js route skeletons, seed scripts for municipality boundaries and time periods, OMI and ISTAT ingestion templates, model job placeholders, and a basic map page wired to latest forecast views.

Developer start checklist

- Provision Supabase project and enable PostGIS.
- Run core, raw, mart, model, and admin schema migrations.
- Load municipality boundaries, ISTAT reference codes, and time periods.
- Backfill OMI valuations and transaction volumes, then validate row counts and geography joins.
- Load ISTAT demographics and tourism datasets and build curated marts.
- Create latest published forecast view, even if first populated by baseline model outputs.
- Build map endpoint and render municipality choropleth in the front end.
- Implement walk-forward back-test job and baseline comparisons before promoting the first forecast model.
- Publish methodology page with confidence and data-quality language before external release.